Knowledge and Information Systems (2026) 68:24
https://doi.org/10.1007/s10115-025-02665-2

RESEARCH

Enhancing recommender systems with LLM-extracted explicit features

Shuai Zhao[1] · **Guoquan Jiang**[2]

Received: 8 May 2025 / Revised: 15 November 2025 / Accepted: 5 December 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract
Existing review-based recommender systems (RSs) are constrained by their shallow semantic representation capabilities, whereas large language models (LLMs) have garnered widespread attention due to their strong contextual understanding and semantic analysis abilities. Despite the significant performance achieved by LLM-powered review-based RSs, they face challenges in terms of inadequate inference efficiency, which hinders their ability to meet the low-latency requirements of real-time recommendations. To enhance the performance of RSs with LLMs while maintaining high efficiency, we propose AEFARec (**A**spect-aware **E**xplicit **F**eature **A**ugmentation for Recommendation). In AEFARec, we initially compute aspect-aware explicit features. This involves utilizing LLMs to extract aspects from unstructured reviews and calculate explicit attribute features of items and explicit preference features of users. Subsequently, we devise a hybrid feature fusion network that integrates implicit features with aspect-aware explicit features to generate final rating predictions. Extensive experiments demonstrate the effectiveness of the proposed AEFARec, and efficiency analysis experiments reveal that AEFARec possesses significantly superior inference efficiency compared to LLM-powered RSs, providing an effective solution for real-time recommendation scenarios.

Keywords Large language model · Review-based recommendation · Recommender systems · Collaborative filtering

1 Introduction

Recommender systems (RSs), as a crucial technology in modern information services, have been deeply integrated into various domains such as e-commerce, education, streaming platforms, and social networks [1]. Their core objective is to analyze historical interactions

✉ Guoquan Jiang
jiangguoquan@cnu.edu.cn

Shuai Zhao
zhaoshuai@cmii.chinamobile.com

[1] China Mobile (Chengdu) Information and Telecommunication Technology Co., Ltd, Chengdu 610200, Sichuan, China

[2] Capital Normal University, Beijing 100048, China

Published online: 29 December 2025

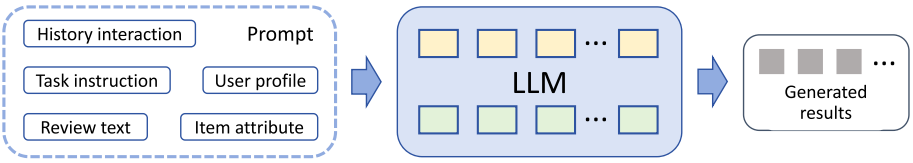


Fig. 1 General process of LLM-powered review-based RSs

between users and items, uncover latent preferences, and provide personalized recommendations to users. Among the methodologies employed in RSs, Collaborative Filtering (CF) has long held a dominant position [2]. The fundamental assumption of CF is that similar users exhibit converging preferences for items, and conversely, similar items are likely to be favored by the same user groups. Classic CF approaches such as MF [3] and NeuMF [4] represent users and items through implicit features, specifically low-dimensional latent vectors, and predict user preferences based on the inner product of these latent vectors. However, these methods rely on the shallow representations of users and items and can only implicitly capture the co-occurrence patterns of user–item interactions, resulting in limited representation capabilities.

To overcome the limitations of CF, researchers explore the integration of user-generated content, particularly user reviews, to enhance the performance of RSs [5]. User reviews naturally contain fine-grained feedback from users about items, including functional evaluations and emotional tendencies, which can reveal explicit preference signals that traditional CF models struggle to capture. Early review-based RSs utilize LDA to analyze review text and apply the results to improve recommendation effectiveness [6, 7]. With the rise of deep learning, researchers employ word vectors to extract semantic information from reviews. For instance, DeepCoNN [8] uses CNN to extract semantic information from reviews for recommendation. SentiRec [9] incorporates sentiment information into review vectors to resolve ambiguities in reviews. NARRE [10] and DAML [11] introduce attention mechanisms into review-based RSs. In recent years, inspired by GNN [12], models such as RGCL [5] and TGNN [13] utilize GNN to model relationships between users and items, while also leveraging reviews to enhance graph learning. However, limited by their shallow semantic representation capabilities, these methods struggle to extract highly discriminative features from complex and diverse reviews and fail to model the dynamic associations between user needs and item attributes.

In recent years, breakthroughs in large language models (LLMs) [14] have provided a novel approach to the in-depth parsing of review texts and the enhancement of recommendation performance [15]. LLMs, represented by GPT-4 [16] and GLM [17], possess powerful contextual understanding, knowledge reasoning, and semantic analysis capabilities. They can mine users' fine-grained sentiments and needs from unstructured reviews. Compared to traditional review-based RSs, the advantages of LLM-powered review-based RSs are: 1) the ability to extract semantic information strongly relevant to the recommendation task without relying on manually defined feature templates; 2) the flexibility to adapt to the specific requirements of different domains and recommendation tasks through fine-tuning or prompt engineering. Figure 1 illustrates the general process of LLM-powered review-based RSs.

Current research on LLM-powered RSs can be primarily categorized into three paradigms. The first category employs prompt learning [18–20], which transforms recommendation tasks into natural language processing tasks by designing appropriate prompts to guide the LLM in generating recommendation results. To better adapt LLMs to specific recommendation

tasks, the second category utilizes fine-tuning approaches that update model parameters using recommendation data while building upon LLMs [21, 22]. The third category adopts instruction tuning [23, 24], where the model is fine-tuned using datasets containing task instructions and exemplars.

However, existing methods share a common critical challenge: they all require feeding various types of information into the LLM for end-to-end inference, such as user profiles, item attributes, history interactions, review texts, and task instructions. Although LLMs demonstrate remarkable advantages in semantic understanding and contextual modeling, their massive parameter sizes and autoregressive generation mechanisms lead to insufficient inference efficiency [25]. While recent advancements in efficient inference [26–28], such as specialized serving frameworks like vLLM, have significantly improved throughput, the inherent latency of large model generation still struggles to meet the stringent requirements of real-time recommendation scenarios for low latency [29–31]. Consequently, devising an efficient architecture that can fully preserve the LLM's semantic analysis capabilities while significantly reducing inference time has become a crucial research problem.

In this paper, we propose AEFARec (**A**spect-aware **E**xplicit **F**eature **A**ugmentation for Recommendation), which leverages the semantic analysis of LLMs in an offline manner to generate high-quality explicit features, enhancing recommendation performance without compromising the low-latency requirements of real-time inference. In AEFARec, we first utilize LLMs to extract aspects across reviews based on NPMI [32] and compute sentiment scores for each review under each aspect. Subsequently, we calculate aspect-aware explicit features. We aggregate the reviews received by each item and compute the explicit attribute features of the item based on the sentiment scores of each aspect in the reviews, representing the quality scores of the item across various aspects. We also aggregate the reviews written by each user to compute the explicit preference features of the user, representing the user's attention to various aspects. Finally, we propose a hybrid feature fusion network, which integrates the implicit features of users and items with the aspect-aware explicit features to produce the final predicted rating.

Our contributions can be summarized as follows:

- We propose an aspect-aware explicit feature extraction method based on LLMs, which explicitly constructs user preference features and item attribute features to enhance recommendation performance.
- We devise an efficient hybrid feature fusion network to utilize implicit features and aspect-aware explicit features for predicting user preferences.
- Extensive experiments demonstrate the superior performance of the proposed AEFARec and its significantly higher efficiency compared to existing LLM-powered RSs.

2 Related work

2.1 Review-based recommendation

In the early stages of review-based RSs, semantic information was explicitly extracted from review texts. LARA [6] used LDA to analyze review texts, with the results used to enhance recommendation effectiveness. HFHT [7] associated each product's rating dimensions with the aspect distribution in review texts to enhance both recommendation effectiveness and interpretability. EFM [33] extracted aspects from review corpora to model explicit aspect factors, thereby enhancing recommendation effectiveness.

Subsequently, with the rise of deep learning, researchers began to use embeddings to extract semantic information from reviews. DeepCoNN [8] employed CNNs to extract semantic information from reviews for recommendation purposes. Sentirec [9] integrated sentiment information into review vectors to resolve ambiguities in reviews. NARRE [10] introduced an attention mechanism to distinguish between useful and useless reviews. DAML [11] utilized attention to capture words in reviews that contribute to item attributes and user preferences, learning the mutual semantic relationships between items and users. CF^2 [34] used counterfactual inference to generate more samples for users with few reviews, mitigating data insufficiency. DRRNN [35] employed autoencoders to extract target reviews as labels, simultaneously optimizing both rating and review loss functions.

Inspired by GNNs [12], RMG [36] utilized GNNs to model relationships between users and items, as well as among users and among items. It employed an attention mechanism to model review features, combining relationship features and review features for recommendation. RGCL [5] treated users and items as nodes and reviews as edges, using message passing and aggregation to learn node representations. It enhanced node representations through graph contrastive learning. TGNN [13] constructed topic graphs and rating graphs to model explicit and implicit information of user preferences, respectively. It integrated features learned from topic graphs and rating graphs to predict user preferences.

Although review-based RSs have achieved remarkable results, these methods are limited by the finite representation capabilities of word vectors and cannot understand the content of reviews, leading to the underutilization of information within reviews. In contrast, LLMs possess open-world knowledge and are capable of further mining the information contained in reviews.

2.2 LLM-powered recommendation

In recent years, LLM-powered RSs have garnered increasing attention. These systems are primarily categorized into three types. The first category relies on prompt learning, transforming the recommendation task into a natural language processing task. By designing appropriate prompts, LLMs are guided to generate recommendation results. Among these, P5 [18] innovatively converts user–item interactions, user descriptions, product information, and user reviews into natural language sequences to enhance recommendation effectiveness. It learns different recommendation tasks through pre-training and personalized prompts. Similarly, ChatRec [19] uses user profiles, historical user interactions, item information, and query inputs as prompts for LLMs, enabling the model to return top N recommendations and enhancing interpretability by inquiring about the rationale behind recommendations. Liu et al. [37] further extend LLMs to address issues like cold-start and cross-domain recommendations by inputting the specific recommendation task as a prompt to adapt to different recommendation scenarios. Rec-SAVER [20] employs a CoT [38] prompting strategy to predict user ratings based on historical purchase records, user reviews, and item descriptions and explains why a user might like an item.

LLM-powered RSs relying on prompt learning are limited by LLM's capabilities on specific tasks. To enhance their adaptation to specific tasks, the second category of methods employs fine-tuning, which updates model parameters using recommendation data. TALLRec [21] fine-tunes the LLM to enhance its understanding of instructions and then uses recommendation data to LoRA fine-tune [39] the LLM to improve its understanding of the recommendation task. Li et al. [40] propose prompt distillation, converting discrete prompt templates into continuous prompt vectors, thereby reducing inference time and the model's

reliance on fine-tuning. RDRec [22] utilizes prompt distillation to improve LLM-powered RSs' ability to capture the underlying reasons behind interactions.

Methods based on distillation are susceptible to overfitting; hence, the third category of methods relies on instruction fine-tuning, which involves fine-tuning LLMs using datasets containing task instructions and examples. InstructRec [23] leverages LLMs to generate a large amount of instruction data containing user preferences to fine-tune T5 [41], enabling it to understand user recommendation instructions and generate personalized recommendation results. BIGRec [24] enables LLMs to generate meaningful item descriptions through instruction fine-tuning. Using these generated descriptions, it finds matching items from a real item set to recommend. Yin et al. [42] unifies the index generation of users and items with the sequential recommendation task within an LLM for joint optimization, integrating semantic information with collaborative information. TokenRec [43] proposes a tokenizer that integrates masked vector quantization techniques to incorporate knowledge from collaborative filtering into ID tokens for LLMs and employs LLMs for generative vector retrieval instead of text generation.

Despite the impressive performance of LLM-powered RSs, their excessive parameters and autoregressive generation mechanism lead to insufficient inference efficiency. In this paper, we explore solutions that can leverage the powerful semantic capabilities of LLMs while maintaining high inference efficiency.

2.3 Aspect-based sentiment analysis

Aspect-based sentiment analysis (ABSA) aims to identify specific aspects discussed in texts and determine their corresponding sentiment polarities. A complete ABSA task consists of two sub-tasks: Aspect Term Extraction (ATE) and Aspect Sentiment Classification (ASC) [44]. ATE identifies the specific aspects related to sentiment from the text. Early ATE methods mostly relied on feature engineering and Conditional Random Fields [45]. With the development of deep learning, methods based on pre-trained language models [46] have improved extraction performance [47, 48].

ASC determines the sentiment polarity for a given aspect [49]. Early methods mostly relied on feature-based classifiers combined with sentiment lexicons and syntactic rules [50, 51]. In recent years, researchers have designed various improved methods based on pre-trained language models. Zhang et al. use GNNs to capture dependency information [52]. Chen and Qian incorporate contrastive learning to enhance the discriminability of aspect and context representations [53]. However, these supervised methods mentioned above depend on manually annotated data. In contrast, this paper explores leveraging the zero-shot reasoning capabilities of LLMs to directly perform aspect extraction and sentiment analysis for recommendation.

3 Method

3.1 Overview

In this paper, we introduce AEFARec (**A**spect-aware **E**xplicit **F**eature **A**ugmentation for Recommendation), which leverages the LLM to extract explicit features for enhancing the performance of RSs. The proposed workflow consists of four primary stages: **1)Aspect Extraction:** An LLM is employed to identify several key aspects from the raw review texts.

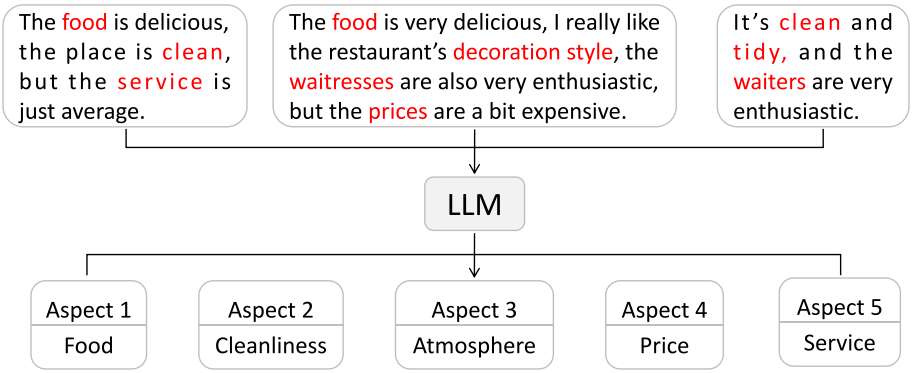


Fig. 2 Aspect extraction

Prompt 1: The uploaded text file is a corpus of <restaurants> review texts. Based on the corpus, summarize the aspects related to <restaurants>. For example, words like <'food'>, <'price'> etc., that can be used to evaluate <restaurants> from different perspectives.
Prompt 2: Understand the text content and determine whether the text mentions the following aspects: <Aspect_1>, ..., <Aspect_k>; output the sentiment score for each aspect, ranging from -1 to 1, where negative numbers indicate dislike, positive numbers indicate like, and 0 indicates neutrality; if an aspectis not mentioned, the sentiment score for that aspect should be 'nan'. Text: <ReviewText>

Fig. 3 The prompt templates employed for aspect extraction and sentiment scoring using LLMs. Prompt 1 is utilized for aspect extraction, while Prompt 2 is employed for sentiment scoring. The content within '<>' can be substituted based on the characteristics of the specific dataset

2)Sentiment Scoring: For each review text, the LLM is utilized to assign sentiment scores to each identified aspect. **3)Explicit Feature Generation:** Based on the sentiment scores of each aspect presented in the review texts, explicit preference features of users toward these aspects and explicit attribute features of items related to these aspects are computed, serving as aspect-aware explicit feature augmentation. **4)Hybrid Feature Fusion:** We devise a hybrid feature fusion network to separately model implicit features and aspect-aware explicit features, and integrate them to obtain the prediction results.

AEFARec fully exploits the semantic analysis capabilities of LLMs to enhance recommendation performance. Moreover, the offline inference approach of LLMs does not impact the online inference speed of AEFARec, thereby achieving remarkable performance while maintaining high inference efficiency.

3.2 Aspect extraction

Extracting aspects from text is a common task in natural language processing. TGNN [13] utilizes BERTopic [54] to encode sentences and then employs Infomap [55] to cluster semantically similar sentences into different aspects. However, this method requires multiple iterations during the clustering process. Using an LLM for aspect extraction can not only leverage the powerful semantic analysis capabilities to perform zero-shot generation, but also consider context. Therefore, we employ the LLMs to extract aspects. Figure 2 demon-

strates exemplars of the aspect extraction method, where we use restaurant reviews from the Yelp dataset.

Due to the limited input processing capacity of the LLM, we sample a small portion of reviews each time for input into the LLM to obtain a set of aspects. We repeatedly input and take the union of the multiple sets of aspects obtained. Firstly, we randomly sample m groups of review data from the dataset. For the j-th group of reviews, a latent aspect set $T_j = \{t_{j1}, t_{j2}, \ldots\}$ is generated through LLM. The Prompt 1 in Fig. 3 demonstrates the prompt template used when applying this method to restaurant reviews from Yelp. After repeating this process m times, we obtain a candidate aspect pool $T = \cup_{j=1}^{m} T_j$, which ensures the completeness of the aspects.

Subsequently, we refine the initial candidate set T by pruning aspects that contribute less to its overall semantic coherence. We measure this coherence using Normalized Pointwise Mutual Information (NPMI) [32]. The NPMI is defined as:

$$NPMI(t_i, t_j) = \frac{\log \frac{P(t_i, t_j)}{P(t_i)P(t_j)}}{-\log P(t_i, t_j)}, \tag{1}$$

where $P(t_i)$ and $P(t_j)$ are the probabilities of occurrence for each aspect, and $P(t_i, t_j)$ is their joint probability of co-occurrence.

Specifically, we employ an iterative backward elimination process. In each iteration, we provisionally remove every aspect from the current set, one at a time, and calculate avgNPMI of the resulting subset. The avgNPMI is defined as:

$$avgNPMI(T) = \frac{2}{|T| \cdot (|T| - 1)} \sum_{1 \le i < j \le |T|} NPMI(t_i, t_j). \tag{2}$$

If the highest avgNPMI achieved by removing an aspect is greater than the score of the current set, we adopt this smaller, more coherent set and proceed to the next iteration. This process terminates until $|T| = k$, where k is a hyperparameter, or when no further improvement in aveNPMI can be achieved by removing any aspect. The final output of this process is the refined aspect set $T^* = \{t_i\}_{i=1}^{k}$.

3.3 Sentiment scoring

Extracting features from review text can provide richer information. SentiRec [9] trains a network to extract sentiment features from reviews and associates them with ratings. SIFN [56] trains a sentiment learner to extract sentiment features from reviews and incorporates them into rating prediction. In this paper, we utilize a zero-shot predictor based on LLM, which requires no training, possesses abundant open-world knowledge, and can generate sentiment scores under multiple aspects.

For each review r in the dataset, by inputting prompts, the LLM evaluates the sentiment orientation of each review under aspects t_1, ..., t_k. Inspired by [57], we set the sentiment score range to $[-1, 1]$, which facilitates capturing both the polarity and intensity of the sentiment simultaneously through its sign and absolute value. The Prompt 2 in Fig. 3 demonstrates the prompt template employed when using restaurant reviews from Yelp as our case study.

The output consists of per-aspect sentiment scores for each review text, represented as: $\mathbf{s}_r = (s_{r,t_1}, ..., s_{r,t_k})$. When the LLM assigns a negative sentiment score (e.g., $s_{r,\text{food}} < 0$) for a review under the 'food' aspect, this indicates that the model has interpreted the review as expressing negative feedback about the restaurant's food quality. Conversely, a positive

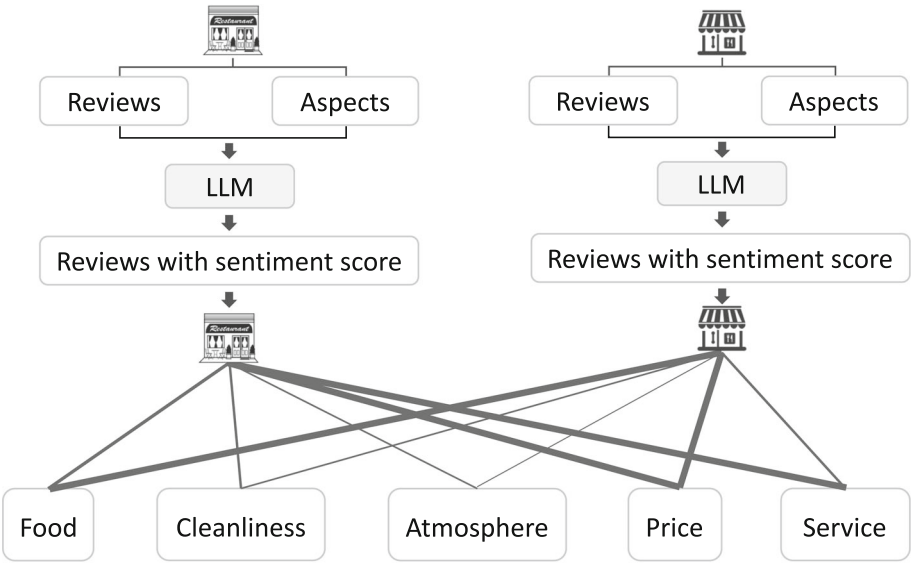


Fig. 4 The construction of explicit attribute features for items

score (e.g., $s_{r,\text{service}} > 0$) under the 'service' aspect suggests a favorable perception of the service. This process ultimately generates comprehensive sentiment scores for each review across aspects $t_1, ..., t_k$.

This sentiment scoring methodology effectively preserves nuanced user preference information through fine-grained continuous sentiment measurements. For instance, a review stating 'the dish was too salty, but the staff were very attentive' would receive scores of -0.8 for the 'food' aspect and 0.9 for the 'service' aspect. These derived scores subsequently serve as the fundamental input for constructing explicit features in later stages.

3.4 Explicit feature generation

Building upon the sentiment scores obtained for each review across aspects, we construct explicit feature vectors for both users and items. For each item i, we first aggregate all its associated reviews into a collection R_i, which represents the set of all reviews received for item i. Inspired by [33], the explicit attribute feature value $f_{i,t}$ for item i on aspect t is computed as:

$$f_{i,t} = \begin{cases} 0, & \text{if i is not reviewed on aspect t,} \\ 1 + \frac{N-1}{1+e^{-n \cdot \bar{s}_{i,t}}}, & else, \end{cases} \tag{3}$$

where n represents the mentioned count of aspect t in review R_i. $\bar{s}_{i,t}$ denotes the average sentiment score of aspect t in review R_i. N is a hyperparameter. This explicit attribute feature calculation method takes into account both the sentiment polarity and the popularity of t. The sigmoid function is employed for nonlinear scaling, which yields smoother results and mitigates the influence of extreme values.

For instance, considering the aspect 'food', the explicit feature $f_{i,t}$ about this aspect represents the rating of item i on the 'food' dimension, reflecting the item's attribute characteristics. Giving k aspects, the explicit attribute feature vector $\mathbf{F}_i \in \mathbb{R}^k$ of the item i is

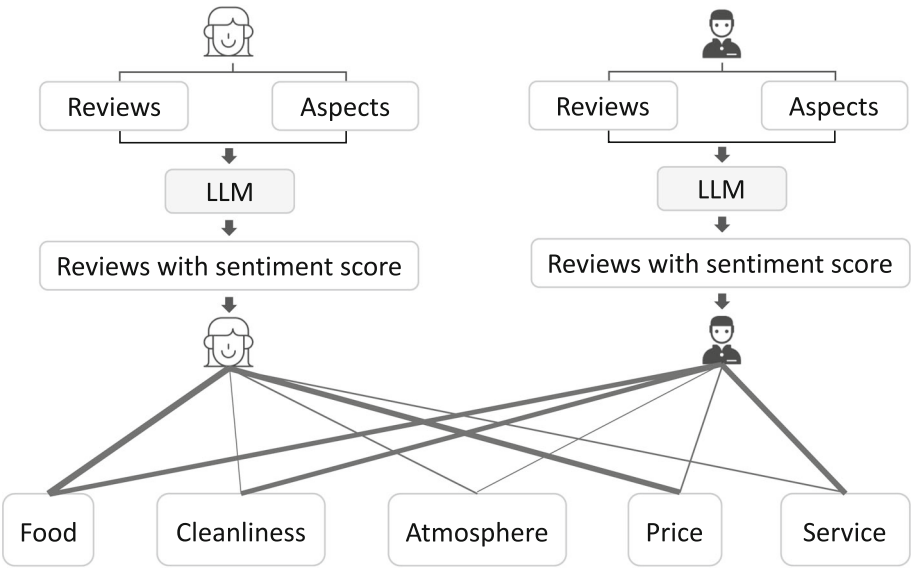


Fig. 5 The construction of explicit preference features for users

constructed by concatenating the attribute feature values of these aspects. Figure 4 illustrates the construction of explicit attribute features for items, where the thicker the line, the higher the rating. The first item has higher ratings in the 'food' and 'service' aspects, while the second item has higher ratings in the 'food' and 'price' aspects.

We also calculate the explicit preference feature vector for each user. Firstly, for user u, we collect the user's reviews R_u, which represents the set of all reviews posted by user u. The explicit preference feature value $f_{u,t}$ for user u on aspect t is computed as:

$$f_{u,t} = \begin{cases} 0, & \text{if i is not reviewed on aspect t,} \\ 1 + (N-1)\left(\frac{2}{1+e^{-c_{u,t}}} - 1\right), & else, \end{cases} \tag{4}$$

where $c_{u,t}$ denotes the mentioned count of aspect t by user u in review R_u. The benefit of this calculation is that it reflects the attention of u to aspect t through $c_{u,t}$. The more frequently an aspect is mentioned, the higher the attention. Additionally, a sigmoid function is employed for nonlinear scaling, resulting in smoother outcomes. Furthermore, this value is scaled to the range of $[1, N]$, facilitating subsequent calculations and analyses.

For example, considering the aspect 'food', the explicit feature $f_{u,t}$ about this aspect represents the preference degree of user u regarding 'food', reflecting the user's attribute characteristics. Similarly, k aspects are selected to compute the explicit features of users. The explicit preference feature vector $\mathbf{F}_u \in \mathbb{R}^k$ of u is constructed by concatenating the user's k preference feature values. Figure 5 illustrates the construction of explicit preference features for users, where the thicker the line, the higher the user's attention. The first user pays more attention to the 'food' and 'price' of the item, while the second user pays more attention to the 'food', 'cleanliness', and 'service' of the item.

This feature construction approach aggregates sentiment scores from multiple reviews to map the aspect-specific sentiment information in review texts into explicit preference features for users and explicit attribute features for items. The resulting features serve as

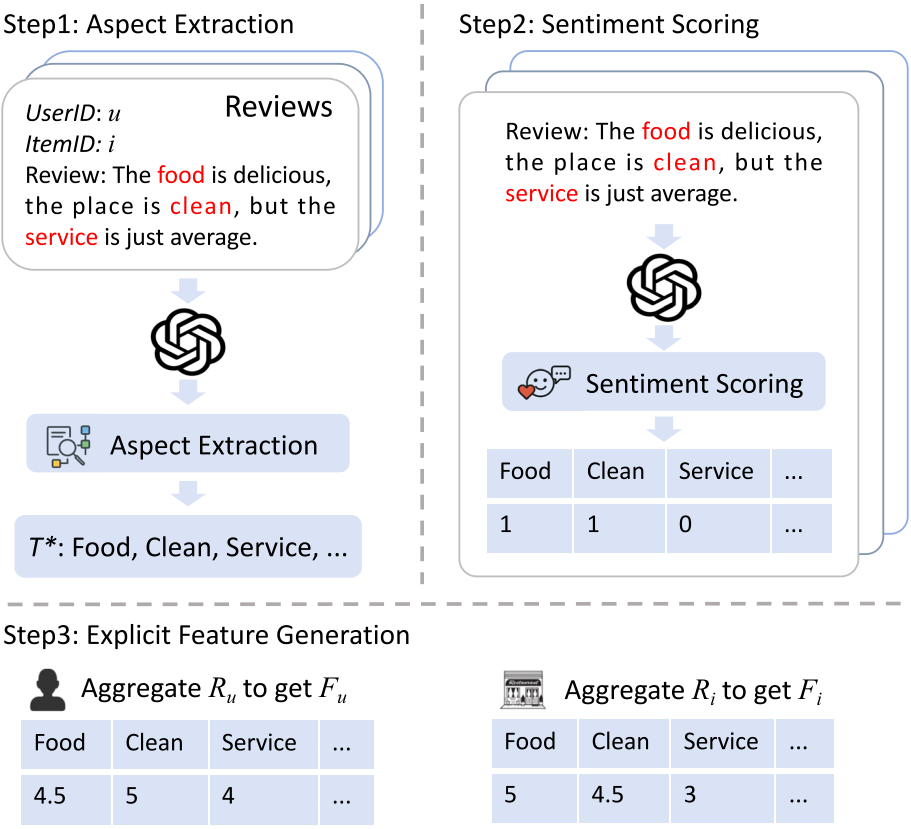


Fig. 6 Overview of constructing the explicit attribute feature vector F_i and the explicit preference feature vector F_u from review text, where R_u denotes the reviews written by user u, and R_i denotes the reviews received for item i

valuable explicit signals to enhance recommendation model performance when modeling user preferences. Figure 6 illustrates the entire process of constructing explicit features.

3.5 Hybrid feature fusion

Leveraging the powerful semantic analysis capabilities of LLMs to obtain explicit features, multiple approaches exist to model these features for enhancing the performance of RSs, such as GNNs or attention mechanisms. To achieve higher inference efficiency, we adopt lightweight models including Factorization Machines (FM) [58] for implicit features and Multi-Layer Perceptrons (MLP) for explicit features, to perform recommendation predictions. As illustrated in Fig. 7, the model architecture consists of two parallel prediction pathways.

Regarding implicit feature interaction modeling, we employ FM to capture latent interaction features between users and items. First, users and items are mapped to d-dimensional implicit feature vectors through an embedding layer. Given a user u and an item i, the formulas

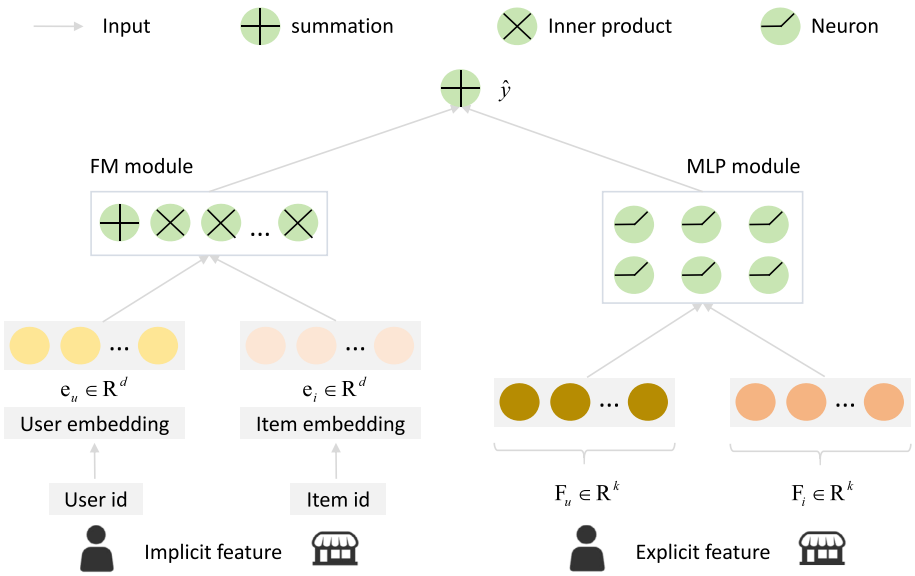


Fig. 7 AEFARec combines implicit features with explicit features extracted from reviews using LLMs to predict user preferences

are as follows:

$$\mathbf{e}_u = \text{Embedding}(u) \in \mathbb{R}^d, \tag{5}$$

$$\mathbf{e}_i = \text{Embedding}(i) \in \mathbb{R}^d. \tag{6}$$

The FM model then computes implicit feature interactions as:

$$y_{\text{fm}} = w_0 + \sum_{j=1}^{d} w_j e_u^{(j)} + \sum_{j=1}^{d} w_j e_i^{(j)} + \sum_{j=1}^{d} \sum_{l=j+1}^{d} \langle \mathbf{v}_j, \mathbf{v}_l \rangle e_u^{(j)} e_i^{(l)}, \tag{7}$$

where $w_0 \in \mathbb{R}$ represents the global bias term, $w_j \in \mathbb{R}$ denotes the weight for the j-th linear component, and $\mathbf{v}_j \in \mathbb{R}^d$ corresponds to the latent factor vector for the j-th feature.

For explicit feature processing, the user's explicit preference feature vector $\mathbf{F}_u$ and item's explicit feature vector $\mathbf{F}_i$ are concatenated and fed into a p-layer MLP network:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1(\mathbf{F}_u \oplus \mathbf{F}_i) + \mathbf{b}_1), \tag{8}$$

$$\mathbf{h}_l = \text{ReLU}(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l), \quad \text{for } l = 2, 3, \ldots, p-1, \tag{9}$$

$$y_{\text{mlp}} = \mathbf{W}_p \mathbf{h}_{p-1} + \mathbf{b}_p, \tag{10}$$

where $\mathbf{W}_1 \in \mathbb{R}^{2k \times h}$ and $\mathbf{W}_2 \in \mathbb{R}^{h \times 1}$ are weight matrices. h is the hidden layer dimension. ReLU is the activation function.

The final prediction combines both pathways through:

$$\hat{y} = \alpha y_{\text{fm}} + (1 - \alpha) y_{\text{mlp}}, \tag{11}$$

where $\alpha \in [0, 1]$ is a learnable weighting parameter.

The model is trained end-to-end using the MSE loss:

$$\mathcal{L} = \frac{1}{|D|} \sum_{(u,i,r)\in D} (\hat{y}(u,i) - r)^2, \tag{12}$$

where D is the training set, and r denotes ground-truth ratings.

Algorithm 1 AEFARec

Input:
D, hyperparameters, $T = \emptyset$, and LLM (APIs or local deployment)
Output:
Aspect set T^*, $\mathbf{F}_u$, $\mathbf{F}_i$, and recommendation model with trained parameters
for $j = 1$ to m **do** ▷ // Aspect Extraction
  Put a review group from D and a prompt into an LLM to generate T_j
  $T \leftarrow T \cup T_j$
end for
while $|T| > k$ **do**
  Drop $t \in S$ that maximizes avgNPMI($S \setminus \{t\}$) using equation 1, 2
  if avgNPMI($S \setminus \{t^\star\}$) > avgNPMI(S) **then**
    $S \leftarrow S \setminus \{t^\star\}$
  else
    break
  end if
end while
for each review $r \in D$ **do** ▷ // Sentiment Scoring
  Get sentiment scores $\mathbf{s}_r$ using LLM
end for
for each item i **do** ▷ // Users' Explicit Feature Construction
  Calculate $\mathbf{F}_i$ using equation 3
end for
for each user u **do** ▷ // Items' Explicit Feature Construction
  Calculate $\mathbf{F}_u$ using equation 4
end for
for epoch = 1 to max epochs **do** ▷ // Hybrid Model Training
  Compute y_{fm} using FM equation 5, 6, and 7
  Compute y_{mlp} using MLP equation 8, 9 and 10 based on $\mathbf{F}_u$ and $\mathbf{F}_i$
  $\hat{y} \leftarrow \alpha y_{\text{fm}} + (1-\alpha) y_{\text{mlp}}$
  Update parameters using $\nabla\mathcal{L}$ where $\mathcal{L} = (\hat{y} - r)^2$
end for

The hybrid architecture enables the FM component to capture latent CF signals while the MLP component models the explicit features extracted by the LLM. Through end-to-end training, the model automatically learns the optimal way to combine these two types of features. During inference, since explicit features can be precomputed, the architecture maintains efficiency comparable to traditional FM models. With relatively few parameters, this architecture achieves both enhanced recommendation performance by leveraging the LLM's semantic analysis capabilities and maintains high inference efficiency.

4 Experiment

In this section, we conduct experiments to address the following questions:

Table 1 Details of dataset statistics

	Yelp	Digital music	Clothing
Users	11,655	5,541	39,387
Items	34,116	3,568	23,033
Reviews	108,706	64,706	278,677
Density	0.03%	0.33%	0.03%
Average user reviews	9	12	7
Min user reviews	5	1	5
Max user reviews	171	578	136
Average item reviews	3	18	12
Min item reviews	1	4	5
Max item reviews	107	268	441

- **RQ1:** Compared to existing CF-based RSs, review-based RSs, and LLM-powered RSs, what is the performance of the proposed AEFARec?
- **RQ2:** What are the final aspect sets extracted from the three datasets?
- **RQ3:** How each component of AEFARec affects recommendation performance?
- **RQ4:** How to select key hyperparameters in AEFARec, such as k?
- **RQ5:** How does the efficiency of AEFARec compare to LLM-powered RSs?
- **RQ6:** How sensitive is the performance of AEFARec to variations in the phrasing of the LLM prompt for sentiment analysis?
- **RQ7:** For which rating levels does AEFARec produce the largest prediction errors?

4.1 Experimental setup

Dataset. We conduct experiments on datasets: Yelp [1] and Amazon .[2] The Yelp dataset comprises data on user reviews, ratings, and other information regarding local businesses, including restaurants, hair salons, and others. Following [33], we utilize review data related to restaurants. Since the raw Yelp dataset is too large, we randomly sampled data from a specific state to constitute the Yelp dataset used in our research. In the Amazon dataset, we select data from two domains: Digital Music and Clothing. Digital Music focuses on the digital music sector and includes user ratings and reviews of songs. Clothing encompasses user reviews and ratings related to apparel and footwear. The ratings in all three datasets are integer values ranging from 1 to 5. Table 1 summarizes the detailed statistics of three datasets, where 'Average user reviews' denotes the average number of reviews per user and 'Average item reviews' denotes the average number of reviews per item.

Evaluation Protocols. Following [18], we use MSE and MAE to evaluate the discrepancy between predicted and actual values, which are two commonly used evaluation metrics in regression problems. MSE computes the average of the squared differences between predicted and actual values, with the formula given by:

$$MSE = \frac{1}{|D|} \sum_{i=1}^{|D|} (y_i - \hat{y}_i)^2, \tag{13}$$

[1] https://business.yelp.com/data/resources/open-dataset/

[2] http://jmcauley.ucsd.edu/data/amazon/

where y_i denotes the actual value, $\hat{y}_i$ represents the predicted value, and $|D|$ signifies the number of samples. MAE calculates the average of the absolute differences between predicted and actual values, with the formula expressed as:

$$MAE = \frac{1}{|D|} \sum_{i=1}^{|D|} |y_i - \hat{y}_i|. \tag{14}$$

MAE directly reflects the absolute magnitude of prediction errors. For the three datasets, each dataset is partitioned into training, validation, and testing sets in the ratio of 8:1:1. The model exhibiting the optimal performance on the validation set is selected for testing on the testing set, and its performance is reported. To minimize unnecessary training iterations, an early stopping strategy [59] is employed, wherein if there is no improvement in the model's performance on the validation set within 10 epochs, the performance of the model on the testing set at that point is considered the final result. To ensure fair comparison and mitigate errors, five experiments are conducted for each method, and the average performance is reported.

Baseline Models. To validate the effectiveness of the proposed AEFARec, we select three types of baseline methods: (1) CF-based RSs, including FM and NeuMF, which are widely used in practice due to their high efficiency. (2) Review-based RSs, such as EFM, DeepCoNN, NARRE, and RGCL. These methods employ various techniques to extract information from review texts to enhance recommendation performance. (3) LLM-powered RSs, which differ from the first two categories by directly inputting item information, user information, review texts, and instructions into the LLM to generate recommendation results. We describe the baseline methods as follows:

- **FM** [58]: FM captures feature interactions through linear combinations of features and second-order cross terms.
- **NueMF** [4]: NueMF integrates matrix factorization with multilayer perceptrons, using neural networks to capture nonlinear user–item interactions.
- **EFM** [33]: EFM is the first to leverage review texts to explicitly model user and item preferences, enhancing the interpretability of the RS.
- **DeepCoNN** [8]: DeepCoNN employs two parallel CNNs to extract features from user and item reviews and then models user preferences toward items through a shared interaction layer.
- **NARRE** [10]: NARRE extracts features from reviews and dynamically weights them using an attention mechanism, simultaneously predicting ratings and generating explainable recommendation rationale.
- **RGCL** [5]: RGCL integrates graph neural networks with contrastive learning to leverage review information for enhancing user preference modeling and rating prediction.
- **P5** [18]: P5 is an LLM-powered RS built on T5 [41], unifying various recommendation tasks under a shared text-to-text framework. It takes as input user ratings and reviews, user/item metadata, and task instructions, and outputs predicted user–item ratings. In this paper, we select P5-S as a baseline model for comparison.
- **TALLRec** [21]: TALLRec is an LLM-powered RS fine-tuned from LLaMA-7B. It processes recommendation data, including user–item ratings, target item descriptions, and task instructions, and employs LoRA to align the LLM with recommendation tasks, improving recommendation performance. The model outputs predicted user–item ratings.

Table 2 Comparison of the performance of various models across three datasets

Model	Digital Music		Yelp		Clothing	
	MSE	MAE	MSE	MAE	MSE	MAE
CF-based RSs						
FM	0.8037	0.6982	1.1829	0.8530	1.1843	0.8936
NeuMF	0.8153	0.7187	1.1716	0.8823	1.2112	0.9053
Review-based RSs						
EFM	1.3254	0.9421	1.7611	1.1704	1.5742	1.0367
DeepCoNN	0.8136	0.7170	1.1520	0.8700	1.1964	0.8925
NARRE	0.7892	0.6751	1.1254	0.8431	1.1620	0.8739
RGCL	0.7595	0.6325	1.0330	0.7912	1.0863	0.8178
LLM-powered RSs						
P5	0.7832	0.6749	1.0574	0.8127	1.1267	0.8384
TALLRec	0.7963	0.6824	1.0633	0.8211	1.0931	0.8191
Rec-SAVER	0.7796	0.6678	1.0654	0.8279	1.1015	0.8280
Ours						
AEFARec-GLM	0.7601	0.6313	0.9561**	0.7564**	1.0601**	0.8002**
AEFARec-GPT	**0.7531***	**0.6287**	**0.9429****	**0.7523****	**1.0563****	**0.7981****

Best performance is **bolded**, second-best is underlined, and third-best is double underlined; ** and * denote significance at $p < 0.01$ and $p < 0.05$, respectively, for the improvement of our model over the strongest baseline

- **Rec-SAVER** [60]: Rec-SAVER uses the zero-shot CoT prompting strategy, guiding the LLM to reason based on user history and item information, and then output a predicted rating.

Implementation Details. We set the max training epochs to 50 and evaluate the model's performance on the validation set after each training epoch. Early stopping is triggered if the validation performance does not improve for 10 consecutive epochs, at which point the optimal model is selected and evaluated on the test set. For the Yelp dataset, the minimum number of aspects, k, is set to 7. For the Digital Music dataset, k is set to 5, and for the Clothing dataset, k is set to 6. For all datasets, N is set to 5, as the maximum rating for these datasets is 5.

Additionally, in this study, we leverage LLMs via API calls to support AEFARec. Specifically, we employ two distinct APIs, GPT-4o [3] and GLM-4-Flash ,[4] as representative paid and free LLM APIs, respectively. To improve the efficiency of sentiment analysis for each review during offline processing, we use asynchronous calls when invoking the API.

4.2 Performance analysis (RQ1)

Table 2 summarizes the performance of AEFARec on three datasets. Based on experimental results, we observe that:

- Overall performance of AEFARec: AEFARec demonstrates superior performance across all evaluation metrics compared to baseline models, providing users with more accu-

[3] https://platform.openai.com/docs/models/gpt-4o.

[4] https://bigmodel.cn/dev/activities/free/glm-4-flash.

Table 3 Aspect sets obtained across three datasets

Dataset	Aspect set
Yelp	Food quality, Service, Price, Atmosphere, Wait times, Menu variety, Drinks
Digital music	Musical style, vocal, Lyrics, Emotion, Innovation
Clothing	Comfort, Size, Fabric quality, Style, Price, Durability

rate and personalized recommendations. This enhancement is achieved by AEFARec's capability to leverage LLMs for extracting semantic information from review texts and transforming this information into explicit user and item features, thereby improving the performance.

- Comparison with LLM-powered RSs: AEFARec outperforms LLM-powered RSs, including P5 and TALLRec. While these LLM-powered RSs can process user interaction histories as text for end-to-end modeling, AEFARec's dedicated FM component is more effective at modeling these latent collaborative signals. By combining these specialized collaborative signals with the explicit features to model user preferences, AEFARec achieves better recommendation quality.
- Comparison with FM: AEFARec exhibits better performance than FM. It is worth noting that in AEFARec, FM is responsible for modeling implicit features of users and items, while MLP models their explicit features. Removing the explicit feature modeling component from AEFARec essentially reduces it to the FM baseline. Thus, the performance comparison between AEFARec and FM serves as an ablation study, demonstrating the benefits of explicit feature modeling. Furthermore, the performance improvement of AEFARec on Digital Music is less pronounced than on Yelp and Clothing. This may be attributed to the less distinctive features of music compared to restaurants and clothing, potentially limiting the effectiveness of explicit feature modeling.
- Comparison between AEFARec-GPT and AEFARec-GLM: We observe that AEFARec-GPT generally outperforms AEFARec-GLM. Here, AEFARec-GPT relies on the GPT-4o API that requires payment, while AEFARec-GLM uses the GLM-4-Flash API that is free. Different LLMs are only employed for the sentiment scoring step. The superior performance of AEFARec-GPT may be attributed to the paid LLM's better performance in sentiment scoring for each aspect, resulting in higher-quality explicit features compared to the free LLM.

4.3 Extracted aspects analysis (RQ2)

The first step in AEFARec involves leveraging an LLM to extract aspects from review texts. During the aspect extraction process, an aspect set $T = \cup_{j=1}^{m} T_j$ is obtained. Subsequently, we employ an iterative optimization algorithm based on NPMI to automatically prune T, with the objective of maximizing the internal semantic coherence of the aspect set while ensuring that the number of aspects is no less than a predefined minimum threshold k. Table 3 presents the final aspect sets derived through this method across the three datasets. These aspect sets are both semantically cohesive and representative of their corresponding domains and are subsequently utilized for constructing explicit features for users and items.

Table 4 Ablation study

Dataset	Method	MSE	MAE
Yelp	AEFARec-GPT	0.9429	0.7523
	AEFARec with simple aspect extraction	0.9828	0.7713
	AEFARec with PyABSA	0.9714	0.7675
	AEFARec with only implicit feature	1.1829	0.8530
	AEFARec with only explicit feature	0.9951	0.7854
Digital Music	AEFARec-GPT	0.7535	0.6287
	AEFARec with simple aspect extraction	0.7642	0.6417
	AEFARec with PyABSA	0.7610	0.6339
	AEFARec with only implicit feature	0.8037	0.6982
	AEFARec with only explicit feature	0.9684	0.7620
Clothing	AEFARec-GPT	1.0563	0.7981
	AEFARec with simple aspect extraction	1.0711	0.8094
	AEFARec with PyABSA	1.0682	0.8045
	AEFARec with only implicit feature	1.1843	0.8936
	AEFARec with only explicit feature	1.1456	0.8505

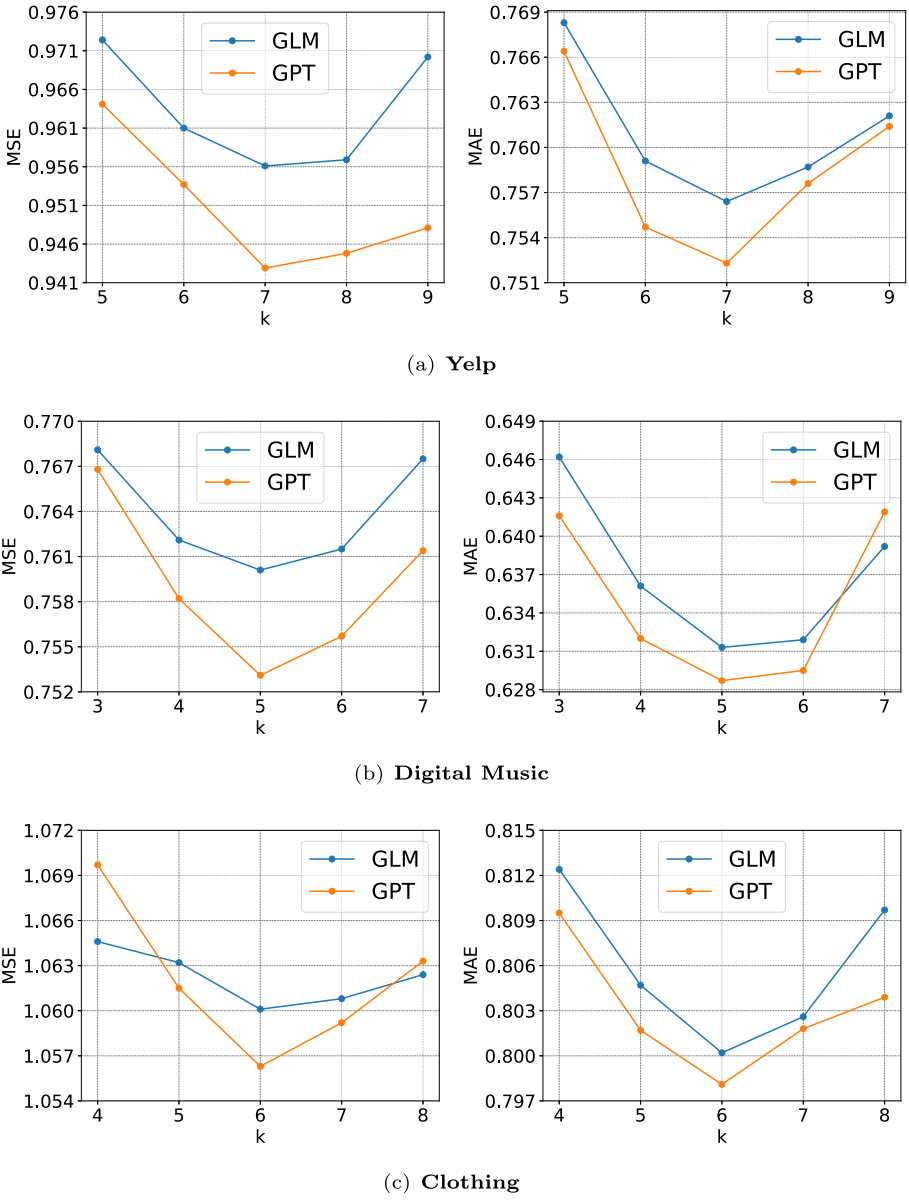
4.4 Ablation study (RQ3)

As shown in Table 4, to validate the effectiveness of each key component in AEFARec, we conduct a series of ablation studies:

- Ablation on aspect extraction: 'AEFARec with simple aspect extraction' indicates that we replace the aspect extraction method in AEFARec with a simplified one-time LLM extraction result. The experiment demonstrates that our proposed NPMI-based aspect extraction method yields superior performance.
- Ablation on sentiment analysis: 'AEFARec with PyABSA' signifies that we replace the LLM-based sentiment analysis module with the well-established open-source library PyABSA [61]. The results indicate that the LLM, leveraging its contextual understanding capability, can extract higher-quality sentiment features beneficial to recommendation.
- Ablation on feature fusion: We evaluate the performance when using only implicit features (with only implicit feature, i.e., the FM model) and only explicit features (with only explicit feature), respectively. The experimental results reveal that the performance of using either type of feature alone is inferior to that of the full AEFARec model. This underscores the effectiveness of the proposed hybrid fusion architecture and the complementarity between explicit semantic features and implicit collaborative signals.

4.5 Hyperparameter analysis (RQ4)

For k: We investigate the impact of k on the model's performance. k denotes the minimum number of aspects obtained during aspect extraction, and it also represents the number of explicit features. Figure 8a illustrates the impact of k on the performance of AEFARec, where 'GLM' refers to the use of GLM-4-Flash, and 'GPT' refers to the use of GPT-4o. As observed in Fig. 8a, when k is too small, the features are insufficient, and there is potential for performance improvement. As k increases, the model's performance improves. However,



(a) **Yelp**

(b) **Digital Music**

(c) **Clothing**

Fig. 8 The impact of k on the performance of the AEFARec model

when k becomes excessively large, some noisy features with weak correlation to user ratings may be introduced, which can degrade overall performance.

For α: To probe the relative importance of implicit signals versus explicit features, we analyze the learned weight parameter α from equation 11. α weights the implicit FM module (y_{fm}), while $(1-\alpha)$ weights the explicit MLP module (y_{mlp}).

Table 5 Comparison of inference times across different models

Model	DeepCoNN	RGCL	P5	TALLRec	Rec-SAVER	AEFARec
Time(s)	5.6×10^{-4}	4.1×10^{-5}	0.29	2.4	1.41	7.6×10^{-6}

Variant 1 of Prompt 2: Paraphrasing
Please review the input text provided below. For each aspect listed (<Aspect_1>, ..., <Aspect_k>), your goal is to identify if it is mentioned. Subsequently, assign a sentiment value to every mentioned aspect based on a scale where 1 signifies a positive feeling, -1 signifies a negative feeling, and 0 is for neutrality. For any aspect not present in the text, you must output 'nan' as its score. Input Text: <ReviewText>
Variant 2 of Prompt 2: Concise Paraphrasing
For the source text below, evaluate the sentiment for these specific aspects: <Aspect_1>, ..., <Aspect_k>. You are required to return a score for each aspect from -1 (negative) to 1 (positive), using 0 for neutral. If an aspect is not discussed, provide 'nan' as the score.

Fig. 9 Prompt variations for the sentiment scoring used in the sensitivity analysis

On the Digital Music, Yelp, and Clothing datasets, the explicit feature weight $(1 - \alpha)$ converged to 0.4215, 0.4366, and 0.4423, respectively.

Consistently, $1 - \alpha < 0.5$ across all datasets, meaning the model automatically assigns a higher weight to the implicit FM module. This indicates that implicit collaborative signals dominate the prediction, while the LLM-extracted explicit features function as an auxiliary signal. This result validates our hybrid method, demonstrating the model's ability to adaptively fuse both heterogeneous information sources for optimal performance.

4.6 Efficiency analysis (RQ5)

We compare the operational speeds of different models on digital music. The GPU we used is the RTX 4070, and the CPU is the i5-12600KF. As shown in Table 5, the inference efficiency of the LLM-powered RS is very low, which cannot meet the efficiency requirements of RSs. The proposed AEFARec has the highest efficiency. This is because, unlike P5 and TALLRec, AEFARec does not require an LLM during inference.

It is worth noting that this architectural choice represents a trade-off: models like P5 and TALLRec offer greater flexibility by handling diverse, unstructured inputs in real time. In contrast, AEFARec intentionally sacrifices this online input flexibility in exchange for substantial gains in recommendation performance and low-latency.

The LLM analyzes reviews offline and outputs explicit features to enhance the performance of the RS. It should be noted that the speed of the LLM in performing sentiment scoring on each review offline is 0.22s for GPT-4o and 0.09s for GLM-4-Flash. For Clothing, which contains more than 278,000 reviews, processing via the GPT-4o API would require over 16 h. However, this process is a one-time preprocessing step and can be significantly accelerated by deploying open-source LLMs locally and leveraging multi-GPU parallel inference. Furthermore, subsequent model updates do not require reprocessing the entire dataset. Only newly generated reviews need to be processed incrementally, and the explicit feature vectors of affected users and items are updated accordingly.

Table 6 Error analysis of AEFARec and FM on different rating samples from Yelp

Dataset	Prompt	MSE	MAE
Yelp	Original prompt	0.9429	0.7523
	Variant 1: Paraphrasing	0.9464	0.7537
	Variant 2: Concise paraphrasing	0.9416	0.754
Digital Music	Original prompt	0.7535	0.6287
	Variant 1: Paraphrasing	0.7528	0.6309
	Variant 2: Concise paraphrasing	0.7551	0.6305
Clothing	Original prompt	1.0563	0.7981
	Variant 1: Paraphrasing	1.054	0.7972
	Variant 2: Concise paraphrasing	1.0534	0.8002

Better performance is indicated in **bold**

4.7 Prompt sensitivity analysis (RQ6)

We conduct a sensitivity analysis experiment on the prompts. The 'Prompt 2' shown in Fig. 3 is used for sentiment scoring. Following [62], we obtain variants of 'Prompt 2', as illustrated in Fig. 9. As can be observed from Table 6, across all datasets, there are no significant differences in the final recommendation performance achieved by using these three prompts with different phrasings. This result indicates that AEFARec is robust to potential fluctuations caused by minor variations in the prompt.

4.8 Error analysis (RQ7)

To investigate the performance of AEFARec on samples with different rating levels, we conduct an error analysis experiment based on GPT-4o. As shown in Table 7, compared to the FM baseline that uses only implicit features, the full AEFARec model performs better on samples with predicted ratings of 1, 4, and 5, but slightly worse on samples with ratings of 2 and 3. Notably, AEFARec shows a greater advantage in predicting samples with extreme ratings, especially those rated 5. This suggests that the explicit sentiment features extracted from reviews are particularly effective at capturing the signals that drive users to give clear-cut evaluations.

We further analyze the model's underperformance on ratings of 2 and 3. We hypothesize that this is related to the design trade-offs in our explicit feature aggregation strategy (equation 3). It captures both the average sentiment polarity $\overline{s}_{i,t}$ and the mention count n, i.e., popularity. This design is effective in reflecting the mainstream opinion and performs well in modeling items with consistent ratings. However, when an item receives polarized reviews (e.g., many 5-star ratings alongside a smaller number of low ratings), n amplifies the influence of the dominant positive reviews, while the minority negative signals are relatively diluted in the averaged sentiment ($\overline{s}_{i,t}$).

Consequently, for these polarizing items, the explicit feature signal is skewed toward the dominant positive sentiment. This leads to over-prediction when the ground truth is ratings of 2 and 3, incurring a higher MSE than the baseline on these specific samples. This reveals a promising future work: exploring finer-grained aggregation strategies that incorporate both the mean and variance of sentiment to improve rating prediction performance for controversial items.

Table 7 Error analysis of AEFARec and FM on different rating samples from Yelp

Rating	1	2	3	4	5	Total
FM	3.6642	**1.9648**	**0.8007**	0.4438	1.1497	1.1829
ARFARec	**3.5261**	2.1443	0.9446	**0.3142**	**0.7765**	**0.9429**

Better performance is indicated in **bold**

5 Conclusion and discussion

In this paper, we investigate how to leverage the powerful semantic analysis capabilities of LLMs to enhance the performance of RSs while maintaining high inference efficiency. We propose AEFARec, which utilizes LLMs to extract aspects and sentiment features from reviews offline, constructing user explicit preference features and item explicit attribute features, and then performs prediction through a lightweight hybrid feature fusion network. This design not only retains the deep semantic analysis advantages of LLMs but also significantly reduces inference latency. Experiments demonstrate that AEFARec outperforms existing methods on multiple datasets while maintaining high inference efficiency.

Despite AEFARec's strong performance, its offline architecture inherently limits data freshness, and it fails to process new reviews in real-time. This limitation, however, is practically manageable. Frequent periodic updates with incremental feature extraction on new reviews can balance timeliness and efficiency. Retraining the lightweight hybrid model is fast (e.g., under 10 min on Clothing), at a fraction of the cost of full reprocessing all datasets (more than 16 h).

In the future, we will investigate how to extract more fine-grained features from reviews to further improve recommendation performance.

Author Contributions Shuai Zhao was responsible for investigation, software, visualization, revision, and writing the draft. Guoquan Jiang was responsible for modification, supervision, and funding acquisition.

Funding This research was funded by the key research projects of the National Language Committee (Grant No. ZDI145-63) and China Foreign Languages Publishing Administration (Project: International Communication in the Context of Generative Artificial Intelligence: Practical Progress and Impact Paths).

Data availability The datasets are publicly available at https://business.yelp.com/data/resources/open-dataset/ and http://jmcauley.ucsd.edu/data/amazon/.

Code Availability Our code is available at https://github.com/uibe-cmcm/AEFARec/tree/main.

Declarations

Conflict of interests The authors declare no conflict of interest.

References

1. Felfernig A, Polat-Erdeniz S, Uran C, Reiterer S, Atas M, Tran TNT, Azzoni P, Kiraly C, Dolui K (2019) An overview of recommender systems in the internet of things. Journal of Intelligent Information Systems 52(2):285–309. https://doi.org/10.1007/s10844-018-0530-7
2. Koren Y, Rendle S, Bell R (2021) Advances in collaborative filtering. Recommender systems handbook, 91–142. https://doi.org/10.1007/978-1-0716-2197-4_3

3. Koren Y, Bell R, Volinsky C (2009) Matrix factorization techniques for recommender systems. Computer 42(8):30–37. https://doi.org/10.1109/MC.2009.263
4. He X, Liao L, Zhang H, Nie L, Hu X, Chua T-S (2017) Neural collaborative filtering. In: Proceedings of the 26th International Conference on World Wide Web, pp 173–182. https://doi.org/10.1145/3038912.3052569
5. Shuai J, Zhang K, Wu L, Sun P, Hong R, Wang M, Li Y (2022) A review-aware graph contrastive learning framework for recommendation. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1283–1293. https://doi.org/10.1145/3477495.3531927
6. Wang H, Lu Y, Zhai C (2011) Latent aspect rating analysis without aspect keyword supervision. In: Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 618–626. https://doi.org/10.1145/2020408.2020505
7. McAuley J, Leskovec J (2013) Hidden factors and hidden topics: understanding rating dimensions with review text. In: Proceedings of the 7th ACM Conference on Recommender Systems, pp 165–172. https://doi.org/10.1145/2507157.2507163
8. Zheng L, Noroozi V, Yu PS (2017) Joint deep modeling of users and items using reviews for recommendation. In: Proceedings of the Tenth ACM International Conference on Web Search and Data Mining, pp 425–434. https://doi.org/10.1145/3018661.3018665
9. Hyun D, Park C, Yang M-C, Song I, Lee J-T, Yu H (2018) Review sentiment-guided scalable deep recommender system. In: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, pp 965–968. https://doi.org/10.1145/3209978.3210111
10. Chen C, Zhang M, Liu Y, Ma S (2018) Neural attentional rating regression with review-level explanations. In: Proceedings of the 2018 World Wide Web Conference, pp 1583–1592. https://doi.org/10.1145/3178876.3186070
11. Liu D, Li J, Du B, Chang J, Gao R (2019) Daml: Dual attention mutual learning between ratings and reviews for item recommendation. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp 344–352. https://doi.org/10.1145/3292500.3330906
12. Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2020) A comprehensive survey on graph neural networks. IEEE transactions on neural networks and learning systems 32(1):4–24. https://doi.org/10.1109/TNNLS.2020.2978386
13. Shuai J, Wu L, Zhang K, Sun P, Hong R, Wang M (2023) Topic-enhanced graph neural networks for extraction-based explainable recommendation. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1188–1197. https://doi.org/10.1145/3539618.3591776
14. Bharathi Mohan G, Prasanna Kumar R, Vishal Krishh P, Keerthinathan A, Lavanya G, Meghana MKU, Sulthana S, Doss S (2024) An analysis of large language models: their impact and potential applications. Knowledge and Information Systems 66(9):5047–5070
15. Zhao Z, Fan W, Li J, Liu Y, Mei X, Wang Y, Wen Z, Wang F, Zhao X, Tang J et al (2024) Recommender systems in the era of large language models (llms). IEEE Trans Knowl Data Eng. https://doi.org/10.1109/TKDE.2024.3392335
16. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al (2023) Gpt-4 technical report. arXiv preprint arXiv:2303.08774. https://doi.org/10.48550/arXiv.2303.08774
17. GLM T, Zeng A, Xu B, Wang B, Zhang C, Yin D, Zhang D, Rojas D, Feng G, Zhao H, et al (2024) Chatglm: A family of large language models from glm-130b to glm-4 all tools. arXiv preprint arXiv:2406.12793. https://doi.org/10.48550/arXiv.2406.12793
18. Geng S, Liu S, Fu Z, Ge Y, Zhang Y (2022) Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In: Proceedings of the 16th ACM Conference on Recommender Systems, pp 299–315. https://doi.org/10.1145/3523227.3546767
19. Gao Y, Sheng T, Xiang Y, Xiong Y, Wang H, Zhang J (2023) Chat-rec: Towards interactive and explainable llms-augmented recommender system. arXiv preprint arXiv:2303.14524. https://doi.org/10.48550/arXiv.2303.14524
20. Tsai A, Kraft A, Jin L, Cai C, Hosseini A, Xu T, Zhang Z, Hong L, Chi E, Yi X (2024) Leveraging llm reasoning enhances personalized recommender systems. In: Findings of the Association for Computational Linguistics ACL 2024, pp 13176–13188. https://doi.org/10.18653/v1/2024.findings-acl.780
21. Bao K, Zhang J, Zhang Y, Wang W, Feng F, He X (2023) Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In: Proceedings of the 17th ACM Conference on Recommender Systems, pp 1007–1014. https://doi.org/10.1145/3604915.3608857

22. Wang X, Cui J, Suzuki Y, Fukumoto F (2024) Rdrec: Rationale distillation for llm-based recommendation. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp 65–74. https://doi.org/10.18653/v1/2024.acl-short.6
23. Zhang J, Xie R, Hou Y, Zhao X, Lin L, Wen J-R (2023) Recommendation as instruction following: A large language model empowered recommendation approach. ACM Transactions on Information Systems. https://doi.org/10.1145/3708882
24. Bao K, Zhang J, Wang W, Zhang Y, Yang Z, Luo Y, Chen C, Feng F, Tian Q (2025) A bi-step grounding paradigm for large language models in recommendation systems. ACM Transactions on Recommender Systems 3(4):1–27. https://doi.org/10.1145/3716393
25. Leviathan Y, Kalman M, Matias Y (2023) Fast inference from transformers via speculative decoding. In: International Conference on Machine Learning, pp 19274–19286. PMLR
26. Tay Y, Tran V, Dehghani M, Ni J, Bahri D, Mehta H, Qin Z, Hui K, Zhao Z, Gupta J et al (2022) Transformer memory as a differentiable search index. Adv Neural Inf Process Syst 35:21831–21843
27. Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez J, Zhang H, Stoica I (2023) Efficient memory management for large language model serving with pagedattention. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp 611–626
28. Joshi T, Saini H, Dhillon N, Maghraoui KE, et al (2025) Paged attention meets flexattention: Unlocking long-context efficiency in deployed inference. arXiv preprint arXiv:2506.07311
29. Lin J, Dai X, Xi Y, Liu W, Chen B, Zhang H, Liu Y, Wu C, Li X, Zhu C et al (2025) How can recommender systems benefit from large language models: A survey. ACM Transactions on Information Systems 43(2):1–47. https://doi.org/10.1145/3678004
30. Yang C, Lin X, Wang W, Li Y, Sun T, Han X, Chua T-S (2025) Earn: Efficient inference acceleration for llm-based generative recommendation by register tokens. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, pp 3483–3494
31. Zhou H, Gu H, Zhan Z, Liu X, Zhou K, Xiao Y, Liang M, Govindan SP, Chawla P, Yang J, et al (2025) The efficiency vs. accuracy trade-off: Optimizing rag-enhanced llm recommender systems using multi-head early exit. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 26443–26458
32. Röder M, Both A, Hinneburg A (2015) Exploring the space of topic coherence measures. In: Proceedings of the Eighth ACM International Conference on Web Search and Data Mining, pp 399–408
33. Zhang Y, Lai G, Zhang M, Zhang Y, Liu Y, Ma S (2014) Explicit factor models for explainable recommendation based on phrase-level sentiment analysis. In: Proceedings of the 37th International ACM SIGIR Conference on Research & Development in Information Retrieval, pp 83–92. https://doi.org/10.1145/2600428.2609579
34. Xiong K, Ye W, Chen X, Zhang Y, Zhao WX, Hu B, Zhang Z, Zhou J (2021) Counterfactual review-based recommendation. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, pp 2231–2240
35. Xi W-D, Huang L, Wang C-D, Zheng Y-Y, Lai J-H (2021) Deep rating and review neural network for item recommendation. IEEE Transactions on Neural Networks and Learning Systems 33(11):6726–6736. https://doi.org/10.1109/TNNLS.2021.3083264
36. Wu C, Wu F, Qi T, Ge S, Huang Y, Xie X (2019) Reviews meet graphs: Enhancing user and item representations for recommendation with hierarchical attentive graph neural network. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 4884–4893. https://doi.org/10.18653/v1/D19-1494
37. Liu J, Liu C, Zhou P, Lv R, Zhou K, Zhang Y (2023) Is chatgpt a good recommender? a preliminary study. arXiv preprint arXiv:2304.10149. https://doi.org/10.48550/arXiv.2304.10149
38. Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24824–24837
39. Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W et al (2022) Lora: Low-rank adaptation of large language models. ICLR 1(2):3
40. Li L, Zhang Y, Chen L (2023) Prompt distillation for efficient llm-based recommendation. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, pp 1348–1357. https://doi.org/10.1145/3583780.3615017
41. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(140):1–67
42. Yin J, Zeng Z, Li M, Yan H, Li C, Han W, Zhang J, Liu R, Sun H, Deng W, et al (2025) Unleash llms potential for sequential recommendation by coordinating dual dynamic index mechanism. In: Proceedings of the ACM on Web Conference 2025, pp 216–227

43. Qu H, Fan W, Zhao Z, Li Q (2025) Tokenrec: Learning to tokenize id for llm-based generative recommendations. IEEE Transactions on Knowledge and Data Engineering
44. Zhang W, Li X, Deng Y, Bing L, Lam W (2022) A survey on aspect-based sentiment analysis: Tasks, methods, and challenges. IEEE Trans Knowl Data Eng 35(11):11019–11038
45. Jakob N, Gurevych I (2010) Extracting opinion targets in a single and cross-domain setting with conditional random fields. In: Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing, pp 1035–1045
46. Devlin J, Chang M-W, Lee K, Toutanova K (2019) Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (long and Short Papers), pp 4171–4186
47. Shi J, Li W, Bai Q, Ito T (2022) Beeae: effective aspect term extraction with artificial bee colony. Journal of Supercomputing **78**(16)
48. Liu C, Hong Y, Xu Q, Yao J (2024) Wke: Word-level knowledge enrichment for aspect term extraction. Artificial Neural Networks and Machine Learning – ICANN 2024: 33rd International Conference on Artificial Neural Networks. Lugano, Switzerland, September 17–20, 2024, Proceedings, Part VII. Springer, Berlin, Heidelberg, pp 381–394
49. Pontiki M, Galanis D, Papageorgiou H, Androutsopoulos I, Manandhar S, Al-Smadi M, Al-Ayyoub M, Zhao Y, Qin B, De Clercq O, et al (2016) Semeval-2016 task 5: Aspect based sentiment analysis. In: International Workshop on Semantic Evaluation, pp 19–30
50. Wilson T, Wiebe J, Hoffmann P (2005) Recognizing contextual polarity in phrase-level sentiment analysis. In: Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing, pp 347–354
51. Jiang L, Yu M, Zhou M, Liu X, Zhao T (2011) Target-dependent twitter sentiment classification. In: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pp 151–160
52. Zhang C, Li Q, Song D (2019) Aspect-based sentiment classification with aspect-specific graph convolutional networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 4568–4578
53. Chen Z, Qian T (2020) Relation-aware collaborative learning for unified aspect-based sentiment analysis. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp 3685–3694
54. Grootendorst M (2022) Bertopic: Neural topic modeling with a class-based tf-idf procedure. arXiv preprint arXiv:2203.05794. https://doi.org/10.48550/arXiv.2203.05794
55. Rosvall M, Bergstrom CT (2008) Maps of random walks on complex networks reveal community structure. Proc Natl Acad Sci 105(4):1118–1123. https://doi.org/10.1073/pnas.0706851105
56. Zhang K, Qian H, Liu Q, Zhang Z, Zhou J, Ma J, Chen E (2021) Sifn: A sentiment-aware interactive fusion network for review-based item recommendation. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, pp 3627–3631. https://doi.org/10.1145/3459637.3482181
57. Hutto C, Gilbert E (2014) Vader: A parsimonious rule-based model for sentiment analysis of social media text. In: Proceedings of the International AAAI Conference on Web and Social Media, vol 8, pp 216–225
58. Rendle S (2010) Factorization machines. In: 2010 IEEE International Conference on Data Mining, pp 995–1000. IEEE https://doi.org/10.1109/ICDM.2010.127
59. Yao Y, Rosasco L, Caponnetto A (2007) On early stopping in gradient descent learning. Constr Approx 26(2):289–315. https://doi.org/10.1007/s00365-006-0663-2
60. Tsai A, Kraft A, Jin L, Cai C, Hosseini A, Xu T, Zhang Z, Hong L, Chi E, Yi X (2024) Leveraging llm reasoning enhances personalized recommender systems. In: Findings of the Association for Computational Linguistics ACL 2024, pp 13176–13188
61. Yang H, Zhang C, Li K (2023) Pyabsa: A modularized framework for reproducible aspect-based sentiment analysis. In: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, pp 5117–5122
62. Chatterjee A, Renduchintala HK, Bhatia S, Chakraborty T (2024) Posix: A prompt sensitivity index for large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024, pp 14550–14565

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Shuai Zhao holds a Master's degree in International Chinese Education from Capital Normal University, as well as a Bachelor's degree in Business Management from the University of International Business and Economics. He conducted research at China Mobile for nine years, and his research interests include artificial intelligence, language intelligence, and generative artificial intelligence.

Guoquan Jiang Professor at the International College of Culture, Capital Normal University, and Researcher at the Chinese Language Intelligence Research Center, is dedicated to research in language intelligence technology, with main research areas in International Chinese Education and language intelligence applications.